How Far Does an Edit Reach?

Repairing Derived Facts in Stale KV Caches

Nour Shreif, Hadi Ghazi, Karin Al Zoabi
Lebanese University

***Abstract*—When a cached document is edited, part of its key-value (KV) cache can still reflect the old text. The model may then answer from outdated information. Mao et al. showed that a short window after the edit repairs nearby dependencies, but the effect is weak when the dependent text is far away. We test a window that runs from the edit to the last dependent sentence. The benchmark uses WikiText-103 passages with inserted factual dependencies and varies distance, hops, fan-out, and paraphrasing. We compare 40 policies on Qwen3-8B, Llama-3.1-8B, and Mistral-7B. For explicit dependencies, an edit-time word index sets the stopping point without another model call and reaches margin recovery of 0.99 to 1.00, with 95 to 100% accuracy on retained items. It recomputes 18 to 50% of the tokens of a full re-read in distant settings. For paraphrased dependencies, combining GPT-4.1 graphs built before and at edit time reaches 0.84 to 0.98. Attention at 512 positions is also effective, but its storage cost is not included. The main difficulty is therefore finding the affected region, rather than recomputing it once it is known.**

***Index Terms*—KV cache, document editing, cache repair, retrieval-augmented generation, ripple effects**

I. Introduction

Retrieval-augmented generation (RAG) systems and agents reuse documents across requests. Keeping the key-value (KV) cache avoids a new prefill for each question. The problem appears when the source document changes. Each cached state is computed from the preceding text, so states after an edit can still represent the old version. A full re-read removes the problem, but it also removes much of the benefit of caching.

Mao et al. [1] compared cache repair policies under fixed recomputation budgets. A contiguous window after the edit, called EditLocal, performed best when the dependent text was nearby. Performance fell when that text was hundreds or thousands of tokens later. The authors proposed extending the window to the dependent text, but did not test it. We test that proposal and study how the end of the window can be found in practice.

Our method recomputes from the edited span to the last dependent sentence. The stopping point is selected for each document. For explicit dependencies, we use an edit-time word index based on the changed words and on rare names and codes followed for up to three hops. This requires no additional model call. Paraphrased dependencies are harder because the wording changes, for example "the sixth plan" and "Plan six". For these cases we test GPT-4.1 dependency graphs built before the edit and queried at edit time. The two sources are complementary: an edit can make one sentence newly relevant and another newly irrelevant, and stale states in either sentence can affect the answer.

We evaluate 40 policies on Qwen3-8B, Llama-3.1-8B, and Mistral-7B. The benchmark uses WikiText-103 passages with controlled factual edits and varies distance, hops, fan-out, distractors, and paraphrasing. The results support three conclusions. First, the path window recovers most of the lost margin when its endpoint is correct. Second, recomputing only the dependent sentences is not always sufficient, because the intervening text can still carry stale information. Third, the choice of finder involves a clear cost and reliability trade-off among the word index, LLM graphs, and attention.

II. Related Work

Repairing stale KV caches. Mao et al. [1] treated cache repair as a budgeted selection problem. They compared contiguous windows, attention-based selection, structural positions, and KV-state deviations. EditLocal was strongest when dependent text was close to the edit. CarrierWindow used the known answer location and avoided the distance problem, but its performance varied by model. Mao et al. suggested recomputation along the path from the edit to the dependent text, without evaluating it. We evaluate that path and focus on how its endpoint can be found without oracle information.

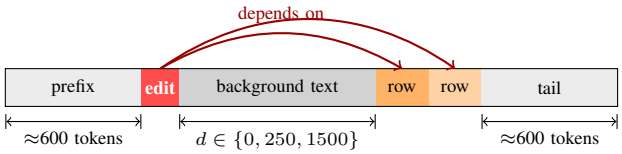
Selective recomputation. PatchKV [3] combines local recomputation with distant blocks selected from stored attention. CacheBlend [4] recomputes selected states to restore interactions between independently cached chunks. Our problem differs because the document was coherent when cached and became stale after a later edit.

Alternative cache-update strategies. Li [2] proposed appending a correction instead of recomputing cached states. Cai and Cai [5] studied related persistence of information in downstream KV states. We test appended corrections across three model families.

Ripple effects of knowledge updates. Knowledge-editing research studies how one changed fact affects related facts [6], including through multi-hop questions [7]. Those studies change model parameters. We study the corresponding problem in a cached document.

III. Benchmark

Benchmark construction. We use real WikiText-103 validation passages [8] as background and insert a small number of synthetic factual sentences. This provides natural surrounding text while giving exact control over the edit, the dependent



Edit: "Company Varnex uses alias Delta" → "…alias Lambda"
Rows: "Alias Delta maps to code Q7"; "Alias Lambda maps to code R6"
Question: Varnex's code? Stale cache: Q7; after the edit: R6.

Fig. 1. What a derived-dependency item looks like. The edit chooses between two lookup rows that are never edited themselves but sit d tokens later. Roughly 600 tokens of ordinary text come before the edit and after the last row, which is what makes stopping early worth anything. The other settings add hops, extra dependent facts, distractors, or paraphrased rows.

sentences, and the expected answer. About 600 tokens precede the edit and about 600 follow the last dependent sentence. The tail is important because it allows a repair to stop before the end of the document. For explicit settings, the background is filtered to remove benchmark names and codes. This prevents unrelated keyword matches, but it also makes the task easier for the word index. We return to this point in the limitations.

The benchmark contains nine settings with 50 candidates each, or 450 candidates per model:

- **Direct dependency:** The edited sentence contains the answer. For example, "Company Varnex's code is Q7" becomes "R6". This tests whether refreshing the edited tokens is sufficient.
- **Derived dependencies:** The edited fact selects between two unchanged lookup rows. For example, "Company Varnex uses alias Delta" becomes "Lambda", while later sentences map Delta and Lambda to different codes. The rows are placed at nominal distances of 0, 250, or 1,500 tokens.
- **Multi-hop dependencies:** The answer requires two lookup steps: alias to group and group to code. The stages are separated by about 250 tokens.
- **Fan-out dependencies:** One alias determines three attributes: code, tag, and room. The question asks for one attribute.
- **Noisy dependencies:** The derived $d = 250$ setting with additional sentences that mention the same company but do not affect the requested attribute.
- **Indirect dependencies:** The dependency is semantic rather than lexical. For example, "the sixth plan" must be linked to "Plan six". These settings use distances of 250 and 1,500 tokens.

The order of each pair of rows is random. This prevents a finder from benefiting from a fixed position and later exposes an important failure in Section VI. Median measured distances are 317 to 362 tokens for the nominal 250 settings and 1,500 to 1,740 tokens for the nominal 1,500 settings. Documents contain about 1,450 to 3,550 tokens.

Edit validity and evaluation filtering. Each edit changes one contiguous span and preserves token length under all three tokenizers. Candidates that do not satisfy this condition are regenerated. Length preservation keeps cached positions aligned and isolates staleness from position shifts caused by insertions or deletions.

Each model keeps an item only if a full re-read improves the new-minus-old margin over the stale cache by at least one nat per token, the full re-read generates the new code, and the stale cache does not already generate it. Of 350 explicit candidates, Qwen3-8B keeps 322, Llama-3.1-8B keeps 263, and Mistral-7B keeps 260. The corresponding kept counts for indirect items are 61, 25, and 32 of 100. The results therefore measure repair on cases that a full re-read can solve and the stale cache cannot. They should not be read as general question-answering accuracy.

IV. Policies and Experimental Setup

A. Repair Policies

Figure 2 summarizes the pipeline. A finder selects the affected positions, the selected cache states are recomputed in place, and the repaired cache is evaluated.

We compare 40 policies. Most selective policies start from the same stale cache and use the same recomputation operator. They differ in the positions selected. Full re-read and appended errata are included as structurally different alternatives. The policies form six groups (Fig. 3):

- **Baselines:** No repair, edited tokens only, recompute from the edit to the end (*suffix*), and full re-read.
- **Fixed windows:** EditLocal [1] with $K \in \{32, 64, 128, 256, 512, 1024\}$ tokens after the edit, and a variant ending at the edited paragraph boundary.
- **Oracle-guided repair:** CarrierWindow recomputes 32 tokens at the known answer-bearing row. Rows only recomputes the dependent sentences. Both use ground-truth locations and serve as diagnostics rather than deployable methods.
- **Dependency-aware path repair:** Recompute from the edit to the furthest identified dependent sentence. The endpoint is supplied by the oracle, word index, LLM graph, or a combination of sources.
- **Alternative strategies:** Appended errata following Li [2], and attention-based selection of individual positions or 32-token blocks following PatchKV [3].
- **Controls:** Random positions or blocks at the path window budget, and StopShort, which stops before the first dependent sentence. These controls separate budget size from position choice and test whether the dependent sentences must be reached.

B. Dependency Identification

The main practical problem is finding the last dependent sentence. We evaluate two approaches.

Edit-time word index. The index compares the old and new edited sentences and collects the changed words. It marks later sentences containing either the removed or introduced terms. This detail matters because the old term identifies a sentence that is no longer relevant but may still contain stale states. From the marked sentences, the index extracts rare names

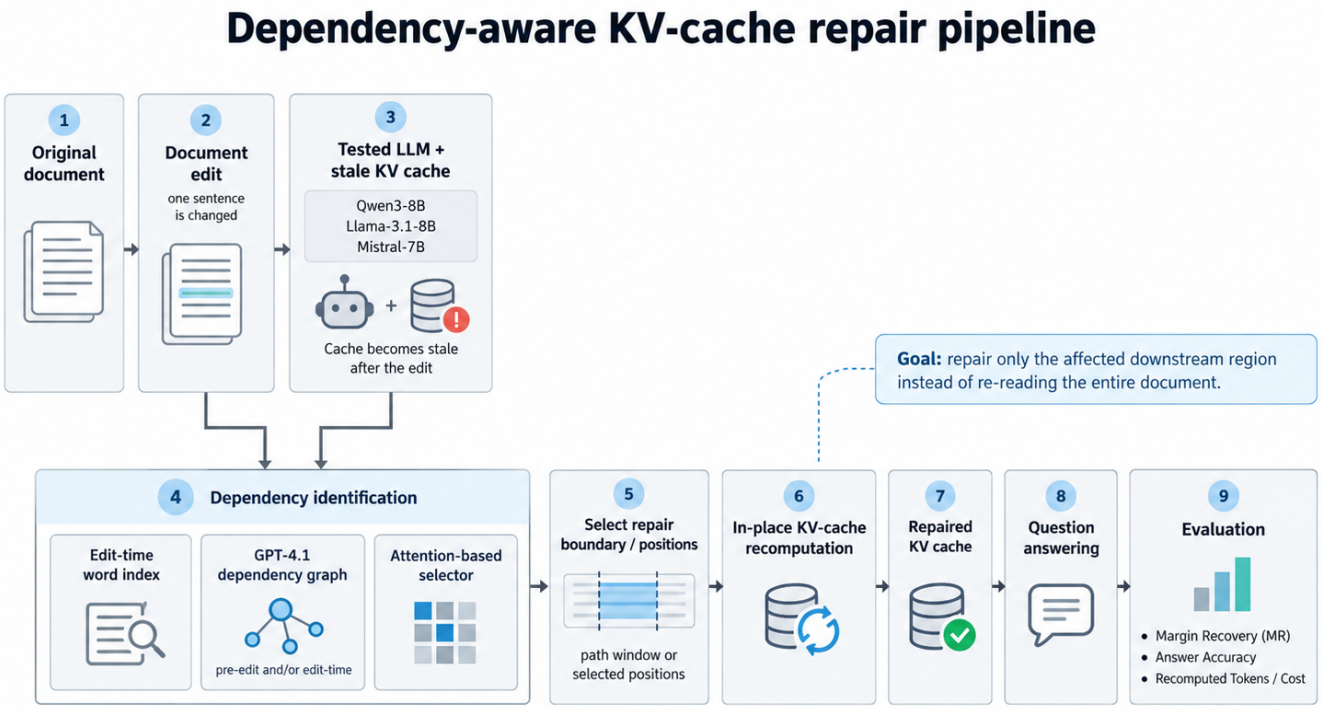


Fig. 2. The repair pipeline, from edit to score. A finder first identifies the sentences affected by the edit. We test a word index, an LLM dependency graph, and attention. The selected positions are then recomputed. The repair is evaluated using margin recovery, answer accuracy, and recomputed tokens.



Fig. 3. One document, ten ways to repair it. Coloured spans are the positions each policy recomputes. EditLocal stops after a fixed count. CarrierWindow and rows-only select the dependent sentences directly. StopShort stops one sentence early. The path window continues to the last dependent sentence. Suffix ignores the endpoint. The erratum appends a correction and recomputes no positions. * marks the oracle diagnostics, which are told where the rows are.

and codes, defined as capitalized terms or tokens containing digits that occur in at most three sentences. It follows these terms for up to three hops and stops at the furthest sentence reached. The method uses no additional model call. It works when dependencies repeat words, but not when the relation is only semantic.

LLM-generated dependency graphs. GPT-4o-mini provides a lower-cost graph builder, and GPT-4.1 tests a stronger model. A graph built before the edit can miss a dependency introduced by the new text. A query at edit time can find the newly relevant sentence but miss the previously relevant one. We therefore also combine both sources and stop at the furthest sentence identified by either one.

C. Models and Implementation

We evaluate Qwen3-8B [9], Llama-3.1-8B-Instruct [10], and Mistral-7B-Instruct-v0.3 [11]. Each model runs in bfloat16 on one NVIDIA L40S GPU. Documents are placed in the user turn of the corresponding chat template, followed by the question.

Repair uses a custom forward pass over a preallocated KV cache. This allows selected positions to be recomputed in place while other cached states are retained. Before the experiments,

the operator is checked against the library implementation. The last-token logits must differ by at most 0.8% of their range and preserve the same top five tokens. Recomputing all positions must also reproduce a fresh prefill.

D. Evaluation Metrics

We report margin recovery, answer accuracy, and recomputation cost.

Margin Recovery (MR). Following Mao et al. [1], let m be the length-averaged log-probability of the new answer minus that of the old answer. Margin recovery is

$$MR = \frac{m_{\text{repaired}} - m_{\text{stale}}}{m_{\text{full}} - m_{\text{stale}}}. \quad (1)$$

A value of zero means no improvement over the stale cache. A value of one means recovery equal to a full re-read. Values outside this range are possible.

Answer Accuracy. Accuracy is the proportion of retained items for which greedy generation, limited to eight tokens, contains the new code and not the old code.

Computational Cost. Cost is measured by recomputed tokens and compared with suffix recomputation and a full re-read. We also record execution time. The cost of LLM dependency extraction and attention-statistic storage is considered separately in Section VII.

Means are reported with 95% confidence intervals from item-level bootstrap resampling [12]. Seven reference policies were repeated on a separate GPU instance and reproduced the original per-item margin recovery values.

V. Results

A. Effectiveness of Dependency-Aware Repair

With the word index selecting the endpoint, the path window reaches margin recovery of 0.99 to 1.00 and answer accuracy of 95 to 100% in all 18 explicit-dependency cells, defined by six settings and three models (Table I). Performance remains high at both short and long distances and in the three-hop, fan-out, and noisy settings.

In these controlled settings, the index matches the oracle path window. It identifies the dependent sentences without selecting unrelated sentences in the filtered explicit benchmark. This result should be interpreted with the benchmark construction in mind, as discussed in the limitations.

For direct dependencies, the answer lies inside the edited sentence. Refreshing only the edited tokens gives margin recovery of 0.91 to 0.96, so a longer window is not needed in that case.

B. Comparison with Fixed-Window Repair

Fixed windows succeed only when the budget reaches the dependent text. EditLocal@32 [1] reaches margin recovery of about 1.00 at $d = 0$, but at most 0.06 in the other distant and multi-hop settings. This agrees with the drop reported by Mao et al. when dependents were moved 250 tokens downstream.

A larger fixed budget does not remove the problem because the required distance varies by document. At the average token cost of the path window, log-interpolated fixed-window recovery is 0.44 to 0.81, compared with about 1.00 for the adaptive window. Even $K = 1024$ does not reach the rows at $d = 1500$, where the measured gap is about 1,500 to 1,740 tokens. Figure 4 shows the same pattern.

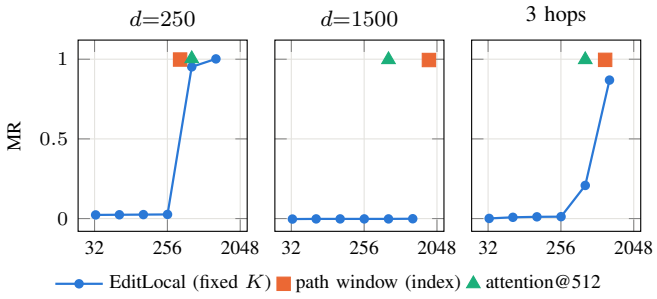


Fig. 4. Recovery against tokens recomputed (log scale), averaged over the three models. The fixed window (line) does nothing until its budget finally reaches the dependent text, then jumps. The path window (single point per panel) spends a different amount on each document because it stops where the dependents end.

C. Computational Efficiency

The path window saves tokens by stopping after the last dependent sentence instead of continuing to the end. In distant settings it recomputes 29 to 65% of the tokens required by suffix recomputation. Relative to a full re-read, it uses about 18% at $d = 250$ and 50% at $d = 1500$. The values are 36% for three hops, 48% for fan-out, and 19% for the noisy setting (Table III). These percentages are token ratios, not direct wall-clock speedups.

D. Comparison with Alternative Repair Policies

CarrierWindow. CarrierWindow recomputes 32 tokens at the known answer-bearing row. It reaches margin recovery of 0.92 to 0.95 on Qwen3-8B for simple derived settings, but only 0.30 to 0.46 on Llama-3.1-8B and 0.56 to 0.69 on Mistral-7B. At three hops, recovery is close to zero on all models. Knowing the answer location is therefore not sufficient when intermediate states also need repair.

Appended corrections. With the edited tokens refreshed, the erratum [2] gives accuracy of 68 to 100% on Llama-3.1-8B, 29 to 54% on Qwen3-8B, and 11 to 43% on Mistral-7B. The method is inexpensive, but its reliability depends strongly on the model.

LLM-generated dependency graphs. The GPT-4o-mini graph built before the edit reaches about 0.84 to 0.85 for adjacent dependencies and 0.09 to 0.53 in the remaining explicit settings. A pre-edit graph can miss the row made relevant by the new text. The word index avoids this specific problem because it follows both removed and introduced terms. Stronger semantic finders are considered for paraphrased dependencies in Section VI.

TABLE I

MARGIN RECOVERY ON THE EXPLICIT DERIVED-DEPENDENCY SETTINGS, AVERAGED OVER RETAINED ITEMS (1 MEANS AS GOOD AS RE-READING THE WHOLE DOCUMENT). EL32 IS EDITLOCAL WITH 32 POSITIONS. CARRIER AND ROWS ARE TOLD WHERE THE ANSWER IS. THE LLM GRAPH IS THE GPT-4O-MINI GRAPH BUILT BEFORE THE EDIT. PATH INDEX, IN BOLD, IS THE WINDOW WHOSE ENDPOINT COMES FROM OUR EDIT-TIME WORD INDEX.

Model	Setting	EL32	Carrier	Rows	Erratum	LLM graph	Random	StopShort	Attn128	Attn512	**Path index**
Qwen3-8B	derived, $d = 0$	1.00	0.94	0.94	0.43	0.85	0.00	0.00	1.00	1.00	**1.00**
	derived, $d = 250$	0.01	0.92	0.92	0.49	0.52	0.05	0.02	0.96	1.00	**1.00**
	derived, $d = 1500$	-0.02	0.93	0.93	0.55	0.26	0.36	-0.02	0.60	1.00	**1.00**
	3 hops	0.02	0.01	0.85	0.24	0.18	0.14	0.03	0.40	1.00	**1.00**
	fan-out 3	0.00	0.50	0.92	0.30	0.19	0.26	0.00	0.56	0.98	**1.00**
	noisy, $d = 250$	0.00	0.95	0.94	0.49	0.09	0.07	0.01	0.85	1.00	**1.00**
Llama-3.1-8B	derived, $d = 0$	1.00	0.46	0.46	0.92	0.84	-0.02	0.00	1.00	1.00	**1.00**
	derived, $d = 250$	0.04	0.30	0.30	1.09	0.53	0.07	0.04	0.92	1.00	**1.00**
	derived, $d = 1500$	0.02	0.37	0.37	1.19	0.30	0.32	0.03	0.65	0.99	**0.99**
	3 hops	0.06	0.04	0.27	0.89	0.29	0.12	0.05	0.48	1.00	**1.00**
	fan-out 3	0.03	0.21	0.32	0.76	0.31	0.17	0.03	0.68	1.00	**1.00**
	noisy, $d = 250$	0.01	0.32	0.32	0.97	0.11	0.06	0.01	0.80	1.00	**0.99**
Mistral-7B	derived, $d = 0$	1.00	0.69	0.69	0.48	0.84	0.00	0.00	1.00	1.00	**1.00**
	derived, $d = 250$	0.02	0.62	0.62	0.48	0.42	0.01	0.02	0.85	1.00	**0.99**
	derived, $d = 1500$	-0.01	0.56	0.56	0.44	0.25	0.29	-0.01	0.62	1.00	**1.00**
	3 hops	-0.07	-0.02	0.56	0.15	0.29	0.00	-0.05	0.35	0.99	**0.99**
	fan-out 3	0.03	0.33	0.66	0.30	0.31	0.21	0.03	0.40	1.00	**1.00**
	noisy, $d = 250$	0.02	0.68	0.68	0.49	0.11	0.07	0.02	0.68	1.00	**1.00**

TABLE II

THE PARAPHRASED SETTINGS, WHERE THE EDIT SAYS "THE SIXTH PLAN" AND THE ROW SAYS "PLAN SIX". MARGIN RECOVERY BY FINDER: THE WORD INDEX HAS NOTHING TO MATCH, GPT-4O-MINI IS TOO WEAK, AND GPT-4.1 ONLY DOES WELL ONCE ITS TWO VIEWS OF THE EDIT ARE COMBINED. n IS THE NUMBER OF RETAINED ITEMS; ACC. IS ACCURACY FOR THAT COMBINED APPROACH.

Model	Distance	n	Word index	GPT-4o-mini pre	GPT-4.1 pre	GPT-4.1 edit	**GPT-4.1 both**	Attn512	Oracle	Acc. (%)
Qwen3-8B	$d = 250$	35	0.32	0.06	0.88	0.55	**0.97**	1.00	1.00	97
Qwen3-8B	$d = 1500$	26	0.42	0.01	0.75	0.61	**0.88**	0.99	1.01	88
Llama-3.1-8B	$d = 250$	15	0.11	0.05	0.86	0.38	**0.93**	1.02	0.99	93
Llama-3.1-8B	$d = 1500$	10	0.39	0.31	0.73	0.71	**0.84**	0.99	1.01	80
Mistral-7B	$d = 250$	13	0.24	-0.11	0.94	0.57	**0.98**	1.03	1.02	85
Mistral-7B	$d = 1500$	19	0.42	0.12	0.91	0.77	**0.93**	0.98	0.98	89

TABLE III

MEAN TOKENS RECOMPUTED ACROSS MODELS, COMPARED WITH SUFFIX AND FULL REREAD. PERCENTAGES ARE TOKEN RATIOS, NOT MEASURED LATENCY.

Setting	Path	Suffix	Full	Path/Full
derived, $d = 0$	20	791	1526	1%
derived, $d = 250$	368	1265	2008	18%
derived, $d = 1500$	1628	2514	3257	50%
3 hops	905	1782	2514	36%
fan-out 3	1512	2391	3142	48%
noisy, $d = 250$	387	1308	2058	19%

TABLE IV

PARAPHRASED-DEPENDENCY MR BY LOOKUP-ROW ORDER, POOLED OVER MODELS AND DISTANCES (61 NEW-ROW-FIRST AND 57 OLD-ROW-FIRST ITEM–MODEL PAIRS).

Finder	New row first	Old row first
GPT-4.1 pre-edit	0.84	0.86
GPT-4.1 edit-time	0.21	1.01
GPT-4.1 both	0.86	1.01
Oracle	0.99	1.01

VI. PARAPHRASED DEPENDENCIES

A. Identifying Semantic Dependencies

The word index is not effective when the dependency is paraphrased. For example, "the sixth plan" and "Plan six" refer to the same plan, but the identifying words differ. The shared word "plan" is too common to provide a reliable link.

The repair itself remains effective. With an oracle endpoint, the path window reaches margin recovery of 0.98 to 1.02 on all three models. With the word index, recovery falls to 0.11 to 0.42 and dependent sentence recall is 0.00 to 0.02. The



Fig. 5. Why neither GPT-4.1 finder is enough on its own (MR from Table IV). Asked at edit time, the model finds the row the edit has just made relevant and stops there. If the new row comes first, the old row is not reached and recovery falls to 0.21. The pre-edit graph has the opposite bias because it is more likely to identify the old row. Using the furthest sentence found by either source covers both orders.

difficulty is therefore locating the dependent sentences.

GPT-4o-mini graphs built before the edit give about -0.11 to 0.31. GPT-4.1 performs better: 0.73 to 0.94 for the pre-edit graph and 0.38 to 0.77 for the edit-time query. Combining both sources gives 0.84 to 0.98, with answer accuracy of 80 to 97% (Table II).

B. Newly Relevant and Previously Relevant Information

A successful repair must cover two groups of sentences: those made relevant by the edit and those made irrelevant by it. The second group is easy to overlook. After a plan change, the old plan row no longer determines the answer, but its cached states were computed when it did. If that row remains outside the window, it can continue to support the old answer.

The two GPT-4.1 sources have opposite weaknesses. The pre-edit graph is more likely to identify the old row and miss the new one. The edit-time query is more likely to identify the new row and miss the old one. Row order determines the effect. GPT-4.1 at edit time gives margin recovery of 1.01 when the old row comes first and 0.21 when the new row comes first (Table IV and Fig. 5). In the second case, the window stops before it reaches the stale old row.

Combining both sources reduces this order effect. Recovery is 0.86 when the new row comes first and 1.01 when the old row comes first, compared with oracle values of 0.99 and 1.01. The dependency set for an edit should therefore include sentences made irrelevant as well as sentences made relevant.

VII. What Repair Requires

A. Repair Location Versus Budget

The controls separate budget size from position choice. Random positions at the path window budget reach margin recovery of at most 0.36. Random 32-token blocks reach at most 0.56. StopShort recomputes the region between the edit and the first dependent sentence but stops before that sentence. It reaches at most 0.05 in distant settings. These results show that a larger budget is not sufficient by itself. The selected positions must include the dependent sentences.

B. Importance of Intervening Context

Recomputing only the dependent sentences gives margin recovery of 0.85 to 0.94 on Qwen3-8B in several derived settings. The corresponding ranges are 0.27 to 0.46 on Llama-3.1-8B and 0.56 to 0.69 on Mistral-7B. The intervening states can therefore remain important, particularly for Llama and Mistral. A contiguous window from the edit to the dependents rebuilds both the dependent sentences and the path between them. This extends the contiguity result of Mao et al. [1] to longer and more complex dependencies.

C. Attention-Based Repair

Attention-based repair selects positions that attended strongly to the edited sentence during prefill. This approach follows PatchKV [3]. At $K = 32$, margin recovery on distant dependencies is at most 0.15. At $K = 128$, it is 0.35 to 0.96. At $K = 512$, it reaches about 0.98 to 1.00 on explicit settings and 0.98 to 1.03 on paraphrased settings. At nominal distance $d = 1500$, attention repair uses a fixed 512 positions, compared with about 1628 tokens for the path window.

These results do not contradict Mao et al. Attention repair is weak at $K = 32$ and stronger at larger budgets. Two practical problems remain. The attention statistics must be stored during prefill or recomputed later, and these costs are not included in the token counts. In addition, $K = 512$ was selected after inspecting the results. Our automatic stopping rules at 25 and 10 percent of maximum attention were not reliable.

D. Practical Efficiency Considerations

The word index requires only a text difference and a scan. A GPT-4.1 request takes on the order of seconds, while path repair takes about 50 ms and a full re-read takes about 200 ms to 225 ms on the tested documents. A GPT-4.1 call at edit time is therefore slower than a full re-read for these documents. The combined finder is best interpreted as evidence about the sentences that must be included, rather than as a low-latency method for short documents. Its relative cost may become more favorable on much longer documents, where a full re-read is more expensive. A graph built when the document is first cached can also reduce the cost at edit time.

VIII. Discussion

The results support a clear interpretation. An edit reaches as far as its last dependent sentence. Recomputing from the edit to that sentence, without gaps, restores the answer in the tested settings. This extends the contiguity result of Mao et al. [1] to longer documents and to derived facts. The random, StopShort, and rows-only controls show that coverage of the path matters more than the number of tokens spent.

The repair operation itself is not the main limitation. Once the endpoint is known, the same method works for explicit and paraphrased dependencies. The practical difficulty is identifying the endpoint. A word index is sufficient for repeated

names and codes, but it does not detect simple paraphrases. LLM-based identification can detect those links, although the most effective combination uses information from before and after the edit and is expensive at edit time. A practical system would therefore use the word index first, build a graph when the document is cached, and reserve an edit-time LLM query for cases in which cheaper methods find no endpoint.

Attention-based repair is competitive at large budgets, especially on long documents and paraphrased settings. Its storage costs and budget selection rule remain unresolved. The repaired cache has also not been shown to be globally consistent. Questions whose answers depend on text after the last identified dependent sentence may still differ from a full re-read.

IX. Limitations

The benchmark facts, edits, and questions are synthetic, although the background text is real. Explicit dependencies repeat names and codes, and the background was filtered so that these terms do not occur in unrelated sentences. The word index results should therefore be interpreted as an upper bound for clean structured text. The paraphrases are also simple. They involve a small number of surface changes and do not include difficult pronoun references, negation, or long ambiguous chains. Some indirect settings contain only 25, 32, or 61 retained items.

The study considers one contiguous and length-preserving edit. Insertions, deletions, and multiple edits are not tested. Documents contain at most about 3550 tokens, and all models have 7 to 8 billion parameters. Results may differ for longer documents and larger models.

The token counts for attention repair do not include the cost of storing or recomputing attention statistics, and $K = 512$ was selected after evaluating several budgets. The GPT-4.1 finder also requires seconds on the tested documents. Finally, evaluation is conditional on the keep rule, and answers are limited to eight greedily generated tokens. The experiments test answer recovery for selected questions. They do not establish full consistency between the repaired cache and a complete re-read.

X. Conclusion

This study examined how far the effect of a document edit extends through a cached document. The results indicate that repair should continue from the edit to the last dependent sentence. For explicit dependencies, a word index identifies this endpoint without an additional model call and gives margin recovery of 0.99 to 1.00 with answer accuracy of 95 to 100 percent on retained cases. When the dependency is paraphrased, combining GPT-4.1 information from before and after the edit gives margin recovery of 0.84 to 0.98.

Repair therefore depends on identifying and covering the affected path. Attention-based repair is competitive at larger budgets, but its storage cost and budget selection rule require further study. Evaluation on naturally occurring revisions and longer documents is the main next step.

Reproducibility

The benchmark construction code, repair implementation, integrity checks, result files, and analysis scripts are available in the project repository:

https://github.com/hadigghazi/kv-edit-repair

References

[1] M. Mao, W. Mackey, and X. Lin, "Contiguity, not importance: Budgeted repair of stale KV caches after document edits," 2026, arXiv:2609.17983.

[2] B. Li, "Models take notes at prefill: KV cache can be editable and composable," 2026, arXiv:2606.17107.

[3] G. Yang, R. Guo, S. He, S. Chen, Y. Hu, and K. Li, "PatchKV: Efficient KV cache recovery for dynamically edited LLM contexts," 2026, arXiv:2609.26219.

[4] J. Yao *et al.*, "CacheBlend: Fast large language model serving for RAG with cached knowledge fusion," in *Proc. European Conference on Computer Systems (EuroSys)*, 2025, arXiv:2405.16444.

[5] Z. Cai and Z. Cai, "Compute globally, materialize locally: The memory contract of sparse event-KV," 2026, arXiv:2607.23693.

[6] R. Cohen, E. Biran, O. Yoran, A. Globerson, and M. Geva, "Evaluating the ripple effects of knowledge editing in language models," *Transactions of the Association for Computational Linguistics*, vol. 12, 2024.

[7] Z. Zhong, Z. Wu, C. D. Manning, C. Potts, and D. Chen, "MQuAKE: Assessing knowledge editing in language models via multi-hop questions," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[8] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," in *Proc. International Conference on Learning Representations (ICLR)*, 2017.

[9] Qwen Team, "Qwen3 technical report," 2025, arXiv:2505.09388.

[10] A. Dubey *et al.*, "The Llama 3 herd of models," 2024, arXiv:2407.21783.

[11] A. Q. Jiang *et al.*, "Mistral 7B," 2023, arXiv:2310.06825.

[12] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.